

Потоки

- Поток - это абстракция последовательности байтов, например
 - **файла,**
 - **устройства ввода-вывода,**
 - **межпроцессного канала связи**
 - **или сокета TCP/IP.**

Операции потока

- Потоки включают три основные операции:
 - **Чтение** из потока. Чтение – это перенос информации из потока в структуру данных, такую как массив байтов.
 - **Запись** в поток. Запись – это перенос данных из структуры данных в поток.

Операции потока

- Потоки могут поддерживать **поиск**. Поиск ссылается на запрос и изменение текущей позиции внутри потока. Поиск возможностей зависит от вида резервного хранилища потока.

Класс Stream

- **Stream** класс и его производные классы предоставляют универсальное представление различных типов ввода и вывода, а также изолируют программиста от специфических особенностей операционной системы и базовых устройств.
- Stream является **абстрактным базовым** классом всех потоков.

Класс Stream

- Можно запрашивать возможности потока с помощью свойств Stream класса: **CanRead**, **CanWrite**, и **CanSeek**.
- В зависимости от базового источника данных или репозитория потоки могут поддерживать только некоторые из этих возможностей.

Класс Stream

- Методы чтения и записи данных в различных форматах: **Read** и **Write**.
- Для потоков, поддерживающих поиск, используются методы **Seek** и **SetLength**.
- Свойства **Position** и **Length**, позволяют запрашивать и изменять текущее положение и длину потока.

Классы для работы с файлами

- Класс FileStream
- Класс BufferedStream
- Классы TextReader и TextWriter
- Классы File и FileInfo
- Классы Directory и DirectoryInfo

FileStream

- Класс, поддерживающий синхронные и **асинхронные** операции чтения и записи.
- **FileStream(String, FileMode)** – инициализирует новый экземпляр класса FileStream указанным путем и режимом создания.
- **FileStream(String, FileMode, FileAccess)** – Инициализирует новый экземпляр класса FileStream заданным путем, режимом создания и разрешением на чтение и запись.

FileMode

- Перечисление, описывающее, каким образом операционная система должна открывать файл.

FileMode

Append	Открывает файл, если он существует, и находит конец файла; либо создает новый файл. (Дозапись в файл)
Create	Указывает, что операционная система должна создавать новый файл. Если файл уже существует, он будет.
CreateNew	Указывает, что операционная система должна создавать новый файл. Если файл уже существует, создается исключение IOException.
Open	Указывает, что операционная система должна открыть существующий файл. Исключение System.IO. FileNotFoundException создается, если файл не существует .
OpenOrCreate	Указывает, что операционная система должна открыть файл, если он существует, в противном случае должен быть создан новый файл.
Truncate	Указывает, что операционная система должна открыть существующий файл. Если файл открыт, он должен быть усечен таким образом, чтобы его размер стал равен нулю байтов.

FileAccess

- Перечисление, определяющее константы чтения, записи или чтения и записи файла.

Read	Доступ для чтения файла. Данные можно прочитать из файла. Для получения доступа для чтения и записи необходимо объединить с Write .
ReadWrite	Доступ для чтения и записи файла . Данные можно записать в файл и прочитать из файла.
Write	Доступ для записи в файл . Данные можно записать в файл. Для получения доступа для чтения и записи комбинируется с Read .

Чтение

- `Read(byte[] array, int offset, int count)` – выполняет чтение блока байтов из потока и запись данных в заданный буфер.
 - `array` – массив байт, в который производится считывание
 - `offset` – смещение в байтах в массиве *array*, в который будут помещены считанные байты.
 - `count` – максимальное число байтов, предназначенных для чтения.

Пример чтения

```
using (FileStream fs = new FileStream(path,
                                     FileMode.Open))
{
    byte[] b = new byte[1024];
    UTF8Encoding temp = new
                                     UTF8Encoding(true);
    while (fs.Read(b, 0, b.Length) > 0)
    {
        Console.WriteLine(temp.GetString(b));
    }
}
```

Оператор using

- Предоставляет удобный синтаксис, обеспечивающий правильное использование объектов **IDisposable**.
- Оператор using соответствующим образом вызывает метод **Dispose** в объекте.
- В рамках блока using объект **доступен только для чтения** и не может быть изменен или переназначен.

Чтения байта

- `ReadByte()` – считывает байт из файла и перемещает положение чтения на один байт.
- Байт приводится к типу **Int32** или **-1**, если достигнут конец потока.

Запись

- `Write(byte[] array, int offset, int count)` – записывает блок байтов в файловый поток.
 - `array` – буфер, содержащий данные для записи в поток
 - `offset` – смещение байтов (начиная с нуля) в объекте `array`, с которого начинается копирование байтов в поток
 - `count` – максимальное число байтов для записи

Пример записи

```
using (FileStream fs = File.Create(path))
{
    info = new
        UTF8Encoding(true).GetBytes("This is some text");
    fs.Write(info, 0, info.Length);
}
```

Запись байта

- `WriteByte(byte value)` – запись байта в текущую позицию в потоке файла.
- `value` – байт, который необходимо записать в поток.

BufferedStream

- Добавляет буферизацию для выполнения операций на другой поток чтения и записи.
- **BufferedStream(Stream)** инициализирует новый экземпляр `BufferedStream` класса размер буфера по умолчанию 4096 байт.
- **BufferedStream(Stream, Int32)**

Пример чтения

```
using (FileStream fs = new FileStream(path,
                                     FileMode.Open))
{
    using (BufferedStream bs = new BufferedStream(fs))
    {
        byte[] b = new byte[1024];
        UTF8Encoding temp = new
                                     UTF8Encoding(true);

        while (bs.Read(b, 0, b.Length) > 0)
        {
            Console.WriteLine(temp.GetString(b));
        }
    }
}
```

StreamReader

- TextReader – абстрактный класс, представляющий средство чтения, позволяющее считывать последовательные **наборы символов**.
- StreamReader – класс, наследник от TextReader, считывает символы из потока байтов в определенной **кодировке**.

Конструкторы

- `StreamReader(String)` – инициализирует новый экземпляр класса `StreamReader` для заданного потока.
- `StreamReader(String, Encoding)` – инициализирует новый экземпляр класса `StreamReader` для заданного потока, используя указанную **кодировку** символов.

Encoding

- Класс Encoding представляет кодировку символов.

|iëàí

1. Éíòðí āāā-òí íà 1500 - ÷òí āāēàòü ñ òàēēìè ìàòāðÿìè è çà÷àì ííè íóæíû
2. ×òí èìàííí ìü òāñòèðóàì: èàēēà ìàòāðè, ÷òí íà ìèõ çà æāēāçí, ïí èàæāíé îòāāēüííé ìàòāðè è èàæāíé îòāāēüííé āēāpōā - èàēēà ðàçúāìû, èàēēà òāðēàēòāðēñòèēē - ïí 2000 íà ìàòü, èàē ÿ ïíèìàp, āñāāí 8000
3. ñíāñòāāííí, ðàçàèò òāñòà ā āēāā òāēñòà - òí÷íāā, èàē áû òāāēēòā ā ñēíāāō - è íáùèà ìùóúāíèÿ îò èñííēüçíāāíèÿ èàæíé ìàòāðè - āāā-òí òüùè ïíèòíðû
4. òāāēēòā - òāñòèì èàæāóp ìàòü ā pc_mark, ÷òí āñòāñòāāííí, āāēüøā, ÷òí íà ìáíāā āñòāñòāāííí - ìùòāāíñÿ ïíñíòðāòü ìà ÿòíí hdtv è fullhd - ÿòí āāā,āāēüøā ìùòāāíñÿ ïíēāðāòü ìà ÿòíí ā ïðíñòāíüēēà èāðû ïðíðēíāí ïíēíēāíèÿ - ÿòí 3. Èāðû ïðíðēíāí ïíēíēāíèÿ - ÿòí òāāā ðāðāòü èàēēà, ÿ ā ìèõ íà ðàçàèðāpñû
5. āûāíā - ðàçāàòü ìàðó ìāāðāā. íāíó çà éó÷øóp ïðíèçāíāèòāēüííñòü, íāíó çà ïòèìāēüííā ñíòííðāíèà òāíà-èà÷āñòāí.

Encoding

ASCII	Получает кодировку для набора символов ASCII (7-разрядных).
Unicode	Получает кодировку для формата UTF-16 с прямым порядком байтов.
UTF32	Получает кодировку для формата UTF-32 с прямым порядком байтов.
UTF7	Получает кодировку для формата UTF-7.
UTF8	Получает кодировку для формата UTF-8.

Пример

```
using(StreamReader sr = new StreamReader(  
    "D:\test.txt", Encoding.ASCII))  
{  
    //код работы с файлом  
}
```

Чтение

- `string ReadLine()` – выполняет чтение **строки** символов из текущего потока и возвращает данные в виде строки.
- Возвращаемое значение – следующая строка из входного потока или значение **null**, если достигнут конец входного потока.

Пример чтения

```
using (StreamReader sr = new StreamReader(  
    "TestFile.txt"))  
{  
    string line;  
    while ((line = sr.ReadLine()) != null)  
    {  
        Console.WriteLine(line);  
    }  
}
```

StreamWriter

- `TextWriter` – абстрактный класс, представляющий модуль записи, который может записывать последовательные наборы символов.
- `StreamWriter` – класс, реализующий `TextWriter` для записи символов в поток в определенной кодировке.

Конструкторы

- `StreamWriter(String)` – инициализирует новый экземпляр класса `StreamWriter` для заданного потока.
- `StreamWriter(String, Encoding)` – инициализирует новый экземпляр класса `StreamWriter` для заданного потока, используя указанную **кодировку** символов.

Запись в файл

- `WriteLine(string value)` – записывает в текстовую строку или поток строку, за которой следует признак конца строки.
- `value` – строка для записи. Если `value` имеет значение **`null`**, записывается только признак конца строки.

Пример записи

```
using (StreamWriter sw = new StreamWriter(  
    "CDriveDirs.txt"))  
{  
    foreach (DirectoryInfo dir in cDirs)  
    {  
        sw.WriteLine(dir.Name);  
    }  
}
```

Класс File

- Предоставляет **статические** методы для **создания, копирования, удаления, перемещения и открытия** одного файла, а также **помогает** при создании объектов FileStream.

Методы

- **Copy**(string sourceFileName, string destFileName)
- Копирует существующий файл в новый файл. Перезапись файла с тем же именем не разрешена.
- **sourceFileName** копируемый файл.
- **destFileName** имя конечного файла. Это не может быть имя каталога или имя существующего файла.

Методы

- FileStream **Create**(string path)
- Создает или перезаписывает файл по указанному пути.
- path – путь и имя создаваемого файла.

```
FileStream fs = File.Create("D:\test.txt");
```

Методы

- **Delete**(string path)
- Удаляет указанный файл.
- path – имя файла, предназначенного для удаления.

Методы

- `bool Exists(string path)`
- Определяет, существует ли заданный файл.
- `path` – проверяемый файл.

```
if (File.Exists("D:\test.txt")) { ... }
```

Методы

- FileStream **Open**(string path, FileMode mode)
- Открывает объект FileStream по указанному пути с доступом для чтения и записи.
- path – открываемый файл.
- mode – значение FileMode.

Методы

- **Move**(string sourceFileName, string destFileName)
- Перемещает заданный файл в новое местоположение и разрешает переименование файла.
- sourceFileName – имя перемещаемого файла.
- destFileName – новый путь к файлу и его имя.

Методы

- Replace(string sourceFileName, string destinationFileName, string destinationBackupFileName)
- Заменяет содержимое **заданного** файла на содержимое **другого** файла, удаляя исходный файл и создавая **резервную копию** замененного файла.

FileInfo

- Предоставляет свойства и методы экземпляра для создания, копирования, удаления, перемещения и открытия файлов, а также позволяет создавать объекты FileStream.
- Этот класс не наследуется.
- `FileInfo fi = new FileInfo(string fileName);`

Свойства

CreationTime	Получает или задает время создания текущего файла или каталога.
Directory	Получает экземпляр родительского каталога.
Exists	Получает значение, показывающее, существует ли файл.
FullName	Получает полный путь к каталогу или файлу.
Length	Получает размер текущего файла в байтах.
Name	Возвращает имя файла.

Методы

- FileInfo CopyTo(string destFileName)
- StreamWriter CreateText()
- void Delete()
- void Encrypt()
- void Decrypt()
- void MoveTo(string destFileName)
- FileStream Open(FileMode mode)

Directory

- Предоставляет **статические** методы для **создания, перемещения и перечисления в каталогах** и вложенных каталогах.
- Этот класс не наследуется.

Методы

- DirectoryInfo **CreateDirectory**(string path)
- Создает все каталоги и подкаталоги по указанному пути, если они еще не существуют.

Методы

- **Delete**(string path)
- Удаляет пустой каталог по заданному пути.

Методы

- `bool Exists( string path )`
- Определяет, указывает ли заданный путь на существующий каталог на диске.

```
if(Directory.Exists(path)) {...}
```

Методы

- string **GetCurrentDirectory()**
- Получает текущий рабочий каталог приложения.

```
string path = Directory.GetCurrentDirectory();
```

Методы

- `string[] GetDirectories( string path )`
- Возвращает имена подкаталогов (включая пути) в указанном каталоге.

Методы

- `string[] GetFiles( string path )`
- Возвращает имена файлов (с указанием пути к ним) в указанном каталоге.

Методы

- `string[] GetFileSystemEntries( string path )`
- Возвращает имена всех файлов и подкаталогов по указанному пути.

Методы

- `string[] GetLogicalDrives()`
- Извлекает имена логических устройств данного компьютера в формате "<имя устройства>:\".

Методы

- DirectoryInfo **GetParent**(string path)
- Извлекает родительский каталог, на который указывает абсолютный или относительный путь.

Методы

- **Move**(string sourceDirName, string destDirName)
- Перемещает файл или каталог со всем его содержимым в новое местоположение.

DirectoryInfo

- Предоставляет методы экземпляра класса для создания, перемещения и перечисления в каталогах и подкаталогах.
- Этот класс не наследуется.

```
DirectoryInfo di = new DirectoryInfo(  
    @"c:\MyDir");
```

Свойства

CreationTime	Получает или задает время создания текущего файла или каталога.
Exists	Получает значение, определяющее наличие каталога.
FullName	Получает полный путь к каталогу или файлу.
Name	Получает имя данного экземпляра DirectoryInfo.
Parent	Получает родительский каталог заданного подкаталога.
Root	Получает корневой элемент пути.

Path

- Выполняет операции для экземпляров класса String, содержащих сведения о пути к файлу или каталогу.
- Эти операции выполняются **межплатформенным** способом.

Поля

- readonly char **PathSeparator**
- Разделитель, задаваемый платформой, который используется в переменных среды для разделения строк пути.
- readonly char **VolumeSeparatorChar**
- Предоставляет разделитель томов, задаваемый платформой.

Методы

- string **Combine**(string[] paths)
- Объединяет массив строк в путь.
- Пустые строки исключаются из объединенного контура.
- Параметры не анализируются, если они имеют пробелы.

Методы

- string **GetDirectoryName**(string path)
- Возвращает для указанной строки пути сведения о каталоге.

Методы

- string **GetFileName**(string path)
- Возвращает имя файла и расширение указанной строки пути.

Методы

- string **GetFullPath**(string path)
- Возвращает для указанной строки пути абсолютный путь.

Методы

- `string GetTempPath()`
- Возвращает путь к временной папке текущего пользователя.

Методы

- Этот метод проверяет существование переменных среды в следующем порядке и использует первый найденный путь:
 - Путь, указанный в переменной среды TMP.
 - Путь, указанный в переменной среды TEMP.
 - Путь, указанный в переменной среды USERPROFILE.
 - Каталог Windows.

Методы

- string **GetTempFileName()**
- Создает на диске временный пустой файл с уникальным именем и возвращает полный путь этого файла.
- Расширение имени файла TMP.
- Временный файл создается во временной папке пользователя, путь к которой, возвращается методом GetTempPath.